

Two-Workflow Release Model

A walkthrough of the workflow_layout example

Jim Fitzpatrick

The problem

Our release workflows report success **before the release is actually complete**.

A single release event triggers a cascade of loosely coupled workflows:

```
Release workflow (reports success)
├── push tag    → image builds (separate workflow)
├── push tag    → CI tests (separate workflow)
├── push branch → more CI tests (separate workflow)
├── push branch → more image builds (separate workflow)
└── release event → helm chart sync
                    └── dispatches to another repo
```

If a downstream workflow fails, **no one is notified** unless they happen to be watching.

What went wrong in v1.5.0

- Tests and image builds ran **in parallel** – images published before tests passed
- Helm chart sync failed for **every component** due to branch protection changes
- Unauthenticated API calls **silently returned empty responses**
- Cross-architecture builds failed with **no recovery mechanism**
- Release workflows reported **success** for all of these

The fix: two workflows

Split every component release into two distinct phases:

Pre-release

- Takes version as input
- Creates release branch if needed
- Makes code changes
- Opens a **pull request**
- Aligns with PR policy

Release

- Triggered after PR merges
- Runs tests **first**
- Builds artifacts sequentially
- Creates GitHub release **last**
- **No code changes allowed**

The release.yaml file

Each component repo has a `release.yaml` that tracks versions:

```
workflow_layout:  
  version: "0.1.0"  
  
dependencies:  
  wasm-shim: "0.2.0"
```

Dependencies can be in two states:

- `"0.0.0"` – targeting latest, used on `main`
- A specific version – targeting a release, used on release branches

This is how we **gate operator releases on operand availability**.

Demo: pre-release workflow

The pre-release workflow is triggered manually with a version input.

It runs three jobs:

- **setup** – validates semver, creates release branch from `main` if needed
- **prepare-release** – updates `release.yaml`, runs component-specific steps, commits and pushes
- **open-pr** – opens a PR to the release branch

Demo: pre-release in action

We triggered pre-release for versions `0.3.0`, `0.4.0`, and `0.5.0`.

The `setup` and `prepare-release` jobs succeeded – release branches were created and pre-release branches pushed.

The `open-pr` job failed:

```
GitHub Actions is not permitted to create or approve pull requests
```

This is an **org-level setting** on the free tier. PRs were created manually instead.

The screenshot shows the GitHub Actions interface for a workflow named 'Pre-release'. The workflow run is labeled 'Pre-release #3' and has a red 'X' icon indicating failure. The left sidebar shows the workflow steps: 'setup' (green checkmark), 'prepare-release' (green checkmark), and 'open-pr' (red X). The main area displays the workflow details for 'pre-release.yaml' on the 'workflow_dispatch' event. It shows a sequence of three jobs: 'setup' (5s), 'prepare-release' (8s), and 'open-pr' (6s). The 'open-pr' job is marked as failed. Below the job sequence, the 'Annotations' section shows '1 error and 3 warnings'. The error for the 'open-pr' job is 'Process completed with exit code 1'.

← Pre-release

Pre-release #3

Summary

All jobs

- ✓ setup
- ✓ prepare-release
- ✗ open-pr

Run details

- Usage
- Workflow file

Manually triggered 34 minutes ago

Status: **Failure**

Total duration: **33s**

Artifacts: -

pre-release.yaml
on: workflow_dispatch

✓ setup 5s → ✓ prepare-release 8s → ✗ open-pr 6s

Annotations
1 error and 3 warnings

✗ open-pr
Process completed with exit code 1.

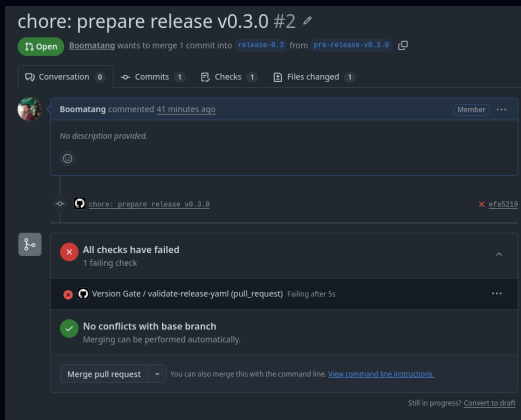
Demo: version gate – blocking a bad release

PR #2 targets `release-0.3` with `release.yaml` declaring a dependency on `wasm-shim: "0.2.0"`.

The **Version Gate** check runs and fails:

```
Dependency 'wasm-shim' targets version '0.2.0',  
but release v0.2.0 does not exist in quadrant-labs/wasm-shim
```

This is the version gating in action – the operator **cannot release** until the operand release exists.



Demo: version gate – passing

PR #3 targets `release-0.4` with a valid `release.yaml`.

The dependency points to a real release.

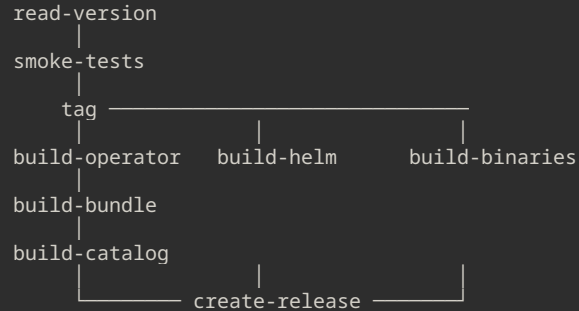
The **Version Gate** check passes.

The screenshot displays a GitHub pull request titled "chore: prepare release v0.4.0 #3". At the top, a green "Open" button is visible, followed by the text "Boomatang wants to merge 2 commits into `release-0.4` from `pre-release-v0.4.0`". Below this, a navigation bar shows "Conversation 0", "Commits 2", "Checks 1", and "Files changed 1". The main content area shows a comment from "Boomatang" 40 minutes ago with the text "No description provided." and a "Member" role indicator. Below the comment, a commit history section shows two commits: "github-actions Bot and others added 2 commits 40 minutes ago" and "chore: prepare release v0.4.0" (commit 3f0e219). A checkmark indicates that the update was verified. At the bottom, a green box with a checkmark icon states "All checks have passed" with "1 successful check". This box contains two specific checks: "Version Gate / validate-release-yaml (pull_request)" which was successful in 5s, and "No conflicts with base branch" which allows for automatic merging. A "Merge pull request" button is present, along with a note that the pull request can also be merged via command line. At the very bottom, a small status bar indicates "Still in progress? Convert to draft".

Demo: the release workflow

After PR #4 (`v0.5.0`) was merged into `release-0.5`, the release workflow was triggered.

The job dependency graph enforces the correct order:

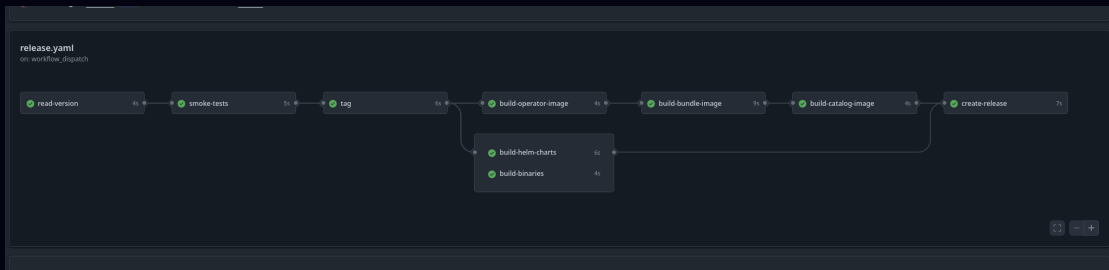


Demo: release workflow results

All 9 jobs completed successfully **in the correct order**:

```
read-version      ✓  
smoke-tests       ✓  
tag               ✓  
build-operator    ✓  
build-helm-charts ✓  
build-binaries    ✓  
build-bundle      ✓  
build-catalog     ✓  
create-release    ✓
```

The GitHub release `v0.5.0` was created **last**, only after every artifact job succeeded.



Demo: the GitHub release

Release v0.5.0 Latest

Compare  

 github-actions released this 38 minutes ago · 0 commits to main since this release  v0.5.0  c646a00 

What's Changed

- Add two-workflow release template by [@Boomatang](#) in [#1](#)
- chore: prepare release v0.5.0 by [@Boomatang](#) in [#4](#)

New Contributors

- [@Boomatang](#) made their first contribution in [#1](#)

Full Changelog: https://github.com/kuadrant-labs/workflow_layout/commits/v0.5.0

Contributors



Boomatang

▼ Assets 2

 Source code (zip) 38 minutes ago

 Source code (tar.gz) 38 minutes ago



 © 2026 GitHub, Inc. [Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#) [Manage cookies](#) [Do not share my personal information](#)

The release was created with `--generate-notes`, pulling commit history automatically.

What this guarantees

- Tests **must pass** before any artifacts are built
- Images build in dependency order: operator → bundle → catalog
- Helm charts synced via **pull request**, not direct push
- The GitHub release is the **last** artifact – if anything fails, there is no release
- **No code changes** during the release workflow
- **No downstream workflow triggers** – everything is contained
- Version gating **prevents** operators releasing before their operands

How sector fits in

Sector moves to a **multi-phase** approach:

1. Trigger **pre-release** workflows for all components – PRs opened for operands and operators
2. Operand PRs merge → sector triggers **release** workflows for operands
3. Operand releases unblock operator PR checks → operator PRs merge
4. Sector triggers **release** workflows for operators and kuadrant-operator

The `release.yaml` version gate is what connects these phases – operator PRs **cannot pass CI** until their operand releases exist.

What's next

- Standardise the two-workflow model across all component repos
- Update sector for multi-phase releases
- Resolve the GitHub Actions PR creation permission on the org
- Replace placeholder steps with real build logic per component
- Add helm chart PR creation with cross-repo token

